

Elementary Algorithms: Correctness and Analysis

Department of Computer Science

Introduction

In this document, we study several basic algorithms. For each algorithm, we present:

- a precise problem statement,
- the algorithm in pseudocode,
- a loop invariant,
- a detailed proof of correctness using the invariant,
- an analysis of running time, and
- space requirements.

The style and terminology follow *Introduction to Algorithms* by Cormen, Leiserson, Rivest, and Stein.

1 Sum of an Array

Problem

Given an array $A[1..n]$, compute the sum of its elements.

Algorithm

ARRAY-SUM(A)

```
1. sum  $\leftarrow$  0
2. for  $i \leftarrow 1$  to  $n$ 
3.     sum  $\leftarrow$  sum +  $A[i]$ 
4. return sum
```

Loop Invariant

At the start of each iteration of the loop with index i ,

$$\text{sum} = \sum_{k=1}^{i-1} A[k].$$

Correctness Proof

Initialization. Before the first iteration ($i = 1$), no elements have been processed. The sum of the empty subarray is 0, and **sum** is initialized to 0, so the invariant holds.

Maintenance. During iteration i , the algorithm adds $A[i]$ to **sum**. Thus after the assignment,

$$\text{sum} = \sum_{k=1}^i A[k],$$

which is exactly the invariant for the next iteration.

Termination. When the loop terminates, $i = n + 1$. By the invariant,

$$\text{sum} = \sum_{k=1}^n A[k],$$

which is the desired result. Therefore, the algorithm is correct.

Running Time

The loop executes n times, each iteration taking constant time.

$$T(n) = \Theta(n).$$

Space Requirements

The algorithm uses constant extra space:

$$\Theta(1).$$

2 Maximum of an Array

Problem

Given an array $A[1..n]$, find the maximum element.

Algorithm

```
ARRAY-MAX(A)
1. max ← A[1]
2. for i ← 2 to n
3.     if A[i] > max
4.         max ← A[i]
5. return max
```

Loop Invariant

At the start of each iteration with index i ,

$$\text{max} = \max\{A[1], A[2], \dots, A[i-1]\}.$$

Correctness Proof

Initialization. Before the loop starts, **max** is set to $A[1]$, which is the maximum of the subarray $A[1..1]$. Thus the invariant holds.

Maintenance. At iteration i , the algorithm compares $A[i]$ with **max**. If $A[i]$ is larger, **max** is updated; otherwise, it remains unchanged. In either case, **max** becomes the maximum of $A[1..i]$, preserving the invariant.

Termination. When the loop terminates at $i = n + 1$, the invariant implies that **max** equals the maximum element of $A[1..n]$. Hence, the algorithm is correct.

Running Time

The algorithm performs $n - 1$ comparisons.

$$T(n) = \Theta(n).$$

Space Requirements

$$\Theta(1).$$

3 Linear Search

Problem

Given an array $A[1..n]$ and a key x , determine whether x appears in the array.

Algorithm

```
LINEAR-SEARCH(A, x)
1. for i <- 1 to n
2.     if A[i] = x
3.         return i
4. return NIL
```

Loop Invariant

At the start of each iteration i ,

$$x \notin A[1..i - 1].$$

Correctness Proof

Initialization. Before the first iteration, no elements have been examined, so x is not in the empty subarray $A[1..0]$. The invariant holds.

Maintenance. If $A[i] \neq x$, the algorithm proceeds to the next iteration, and the invariant continues to hold since x is not in $A[1..i]$.

Termination. If the algorithm returns an index, then $A[i] = x$ and the answer is correct. Otherwise, when the loop terminates, the invariant implies that x does not occur anywhere in the array. Thus the algorithm is correct.

Running Time

Best case: $\Theta(1)$

Worst case: $\Theta(n)$

Space Requirements

$\Theta(1)$.

4 Iterative Binary Search

Problem

Given a sorted array $A[1..n]$ and a key x , find the index of x if it exists.

Algorithm

```
BINARY-SEARCH(A, x)
1. low ← 1, high ← n
2. while low ≤ high
3.     mid ← floor((low + high)/2)
4.     if A[mid] = x return mid
5.     else if A[mid] < x
6.         low ← mid + 1
7.     else high ← mid - 1
8. return NIL
```

Loop Invariant

At the start of each iteration,

if x appears in A , then it lies in $A[\text{low}..\text{high}]$.

Correctness Proof

Initialization. Initially, $\text{low} = 1$ and $\text{high} = n$, so the invariant holds trivially.

Maintenance. Each iteration compares x with $A[\text{mid}]$ and discards half of the search interval where x cannot lie. The remaining interval still contains x if it exists, preserving the invariant.

Termination. If x is found, the correct index is returned. Otherwise, when $\text{low} > \text{high}$, the interval is empty, and the invariant implies that x is not present. Hence, the algorithm is correct.

Running Time

The interval size halves each iteration.

$$T(n) = \Theta(\log n).$$

Space Requirements

$\Theta(1)$.

5 Bubble Sort

Problem

Sort an array $A[1..n]$ into nondecreasing order.

Algorithm

```
BUBBLE-SORT(A)
1. for i <- 1 to n-1
2.     for j <- 1 to n-i
3.         if A[j] > A[j+1]
4.             swap A[j], A[j+1]
```

Loop Invariant

After the i -th pass of the outer loop, the last i elements are in their final sorted positions.

Correctness Proof

Initialization. Before any passes, zero elements are guaranteed to be in their final positions.

Maintenance. Each pass moves the largest remaining unsorted element to its correct position at the end of the array.

Termination. After $n - 1$ passes, all elements are in sorted order. Thus, the algorithm is correct.

Running Time

$$T(n) = \Theta(n^2).$$

Space Requirements

$\Theta(1)$.

6 Selection Sort

Algorithm

```
SELECTION-SORT(A)
1. for i <- 1 to n-1
2.     min <- i
3.     for j <- i+1 to n
4.         if A[j] < A[min]
5.             min <- j
```

6. swap $A[i], A[\min]$

Loop Invariant

At the start of iteration i , the subarray $A[1..i-1]$ contains the $i-1$ smallest elements in sorted order.

Correctness Proof

The inner loop finds the smallest element in the unsorted portion of the array. Swapping it into position i preserves the invariant. When the loop terminates, all elements are sorted.

Running Time

$$T(n) = \Theta(n^2).$$

Space Requirements

$$\Theta(1).$$

7 Matrix Addition

Problem

Given matrices A and B of size $m \times n$, compute $C = A + B$.

Algorithm

```
MATRIX-ADDITION(A, B)
1. for i <- 1 to m
2.   for j <- 1 to n
3.      $C[i][j] \leftarrow A[i][j] + B[i][j]$ 
```

Loop Invariant

At the start of iteration (i, j) , all entries

$$C[r][c] = A[r][c] + B[r][c]$$

for $r < i$ or $(r = i \text{ and } c < j)$ have been correctly computed.

Correctness Proof

Each iteration computes exactly one entry of C . When the loops terminate, all entries have been computed correctly.

Running Time

$$T(m, n) = \Theta(mn).$$

Space Requirements

$$\Theta(mn).$$

8 Brute-Force k -th Largest Element

Problem

Given an array $A[1..n]$, find the k -th largest element.

Algorithm

KTH-LARGEST(A, k)

1. sort A in nondecreasing order
2. return $A[n-k+1]$

Correctness Proof

Sorting arranges elements in increasing order of value. The element at index $n - k + 1$ has exactly $k - 1$ elements larger than it, and therefore is the k -th largest.

Running Time

Using a quadratic-time sorting algorithm:

$$T(n) = \Theta(n^2).$$

Space Requirements

$$\Theta(1).$$